



BITS Pilani

Cloud Computing

SEWP ZG527

AGENDA



01



Containers

02



Introduction & Motivation for Containers

03



Linux Container – LXC and LXD

04



Container Architecture

05



Orchestration technologies

06



Docker Container and Components

07



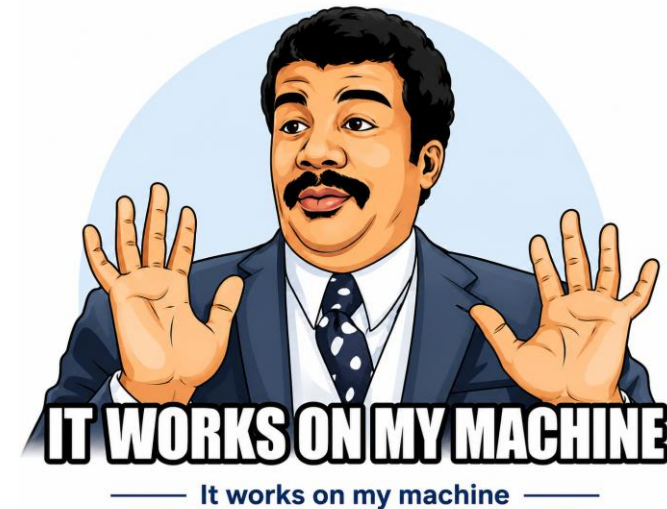
Hands on with Docker



BITS Pilani

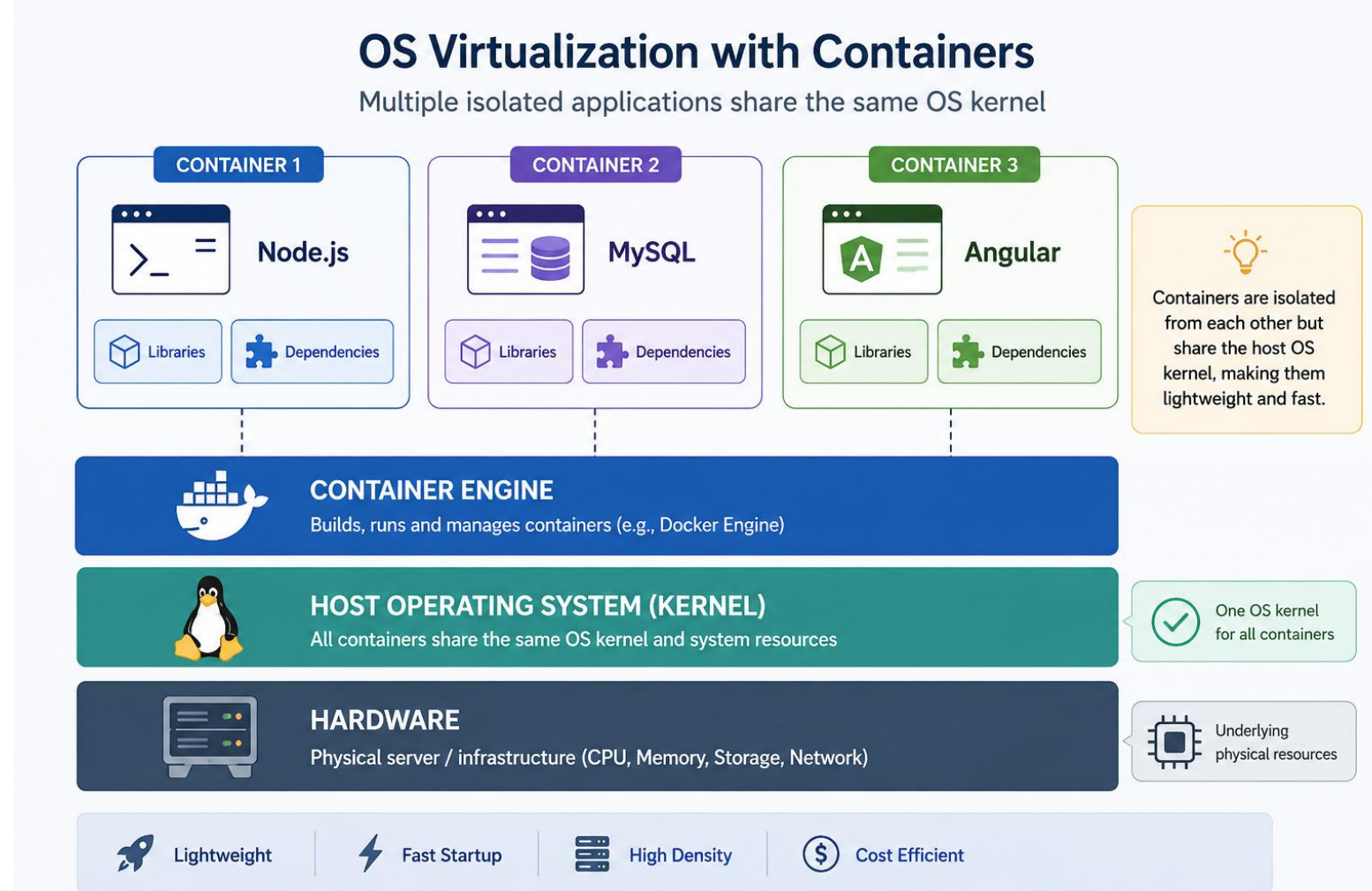
Pilani | Dubai | Goa | Hyderabad

Containers

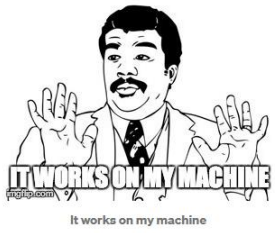


Motivation towards Containers

- Let's assume that you are **building a web service** on an Ubuntu machine. Your code works **fine** on your **local machine**. You have a **remote server** in your data centre that can run your application.
- You **copy** your local binaries on the **remote server** and try to run your code. The next thing that you see is your **code doesn't work** there. The above problems result in **portability issues**. The developer has to spend a **lot of time debugging the environment-specific issues**.



Motivation towards Containers

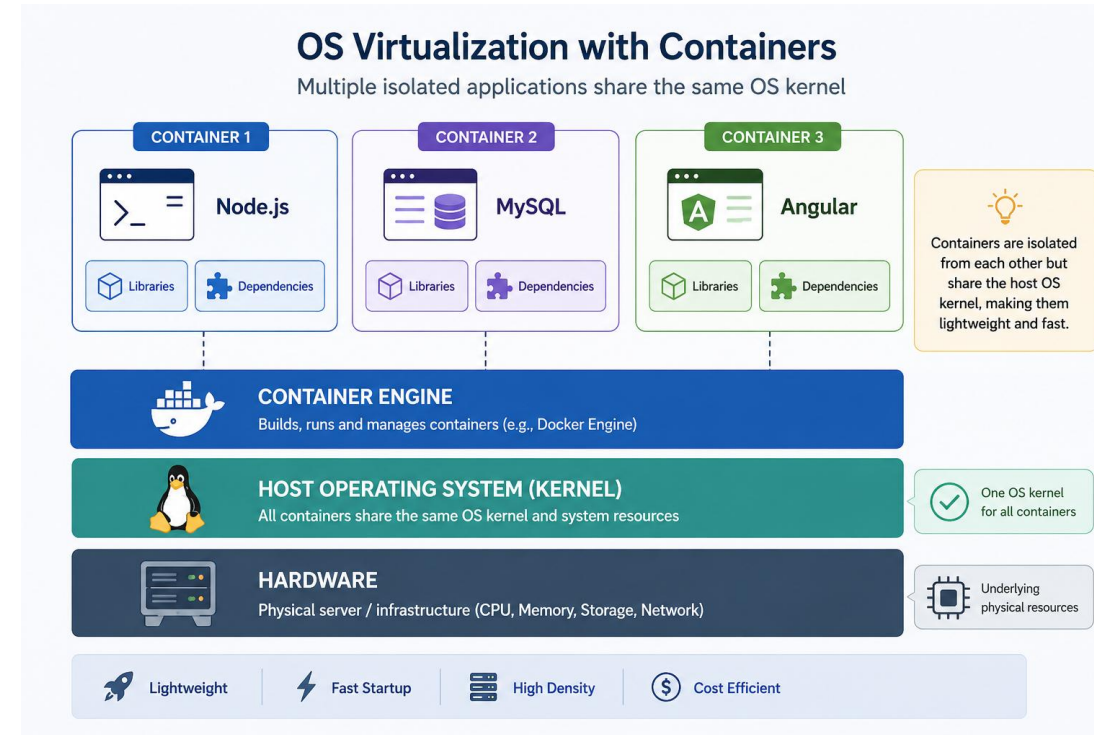


The computer has different hardware resources such as RAM, hard disk, Network Interface Card, IO devices, etc. The operating system is the software that manages this hardware.

OS consists of a **system program known as the kernel**, which is loaded in the memory when OS starts. The kernel is responsible for process management, CPU scheduling, file system & IO.

User programs interact with the hardware through the means of the kernel. For eg:- Let's say your application wants to open a file and write content in it. The application will invoke system calls like `fopen()` and `fwrite()` to perform its functions.

The kernel performs the function on behalf of the user program and gives the output back to it. The following diagram shows the different layers involved in the functioning of an application program.



What are Containers?

A container is a software that packages up application code and all its dependencies so the application can be ported from one computing environment to another.

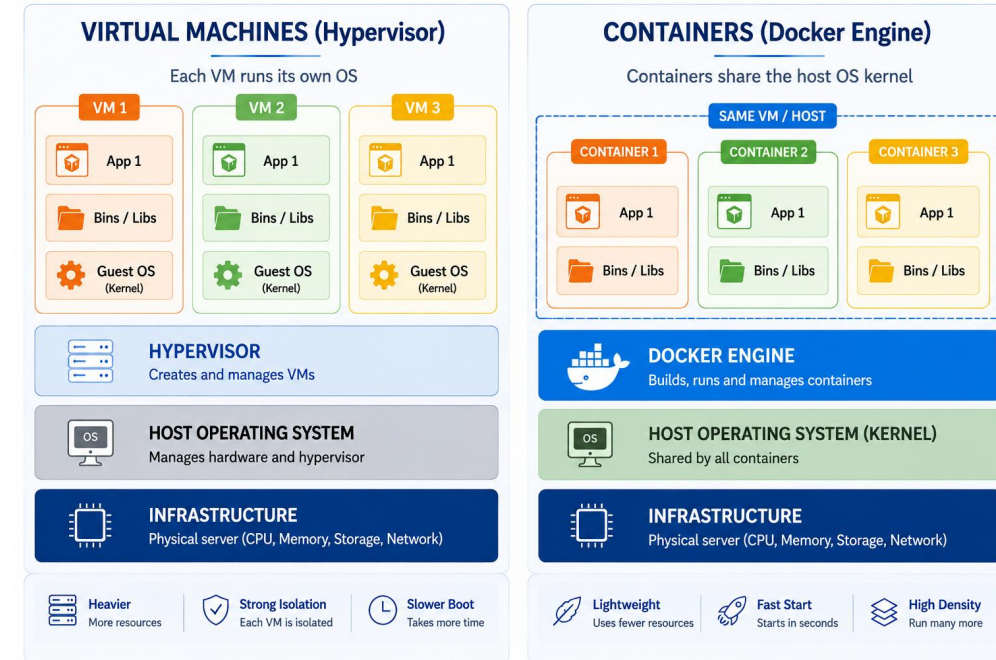
The way which containerized applications operate is shown →

Containers are an **operating system virtualization technology** used to package **applications** and their **dependencies** and run them in **isolated environments**.

They provide a lightweight method of packaging and deploying applications in a standardized way across many different types of infrastructure

Containers run consistently on any container-capable host, so developers can test the same software locally that they will later deploy to full production environments.

The container format also ensures that the application dependencies are baked into the image itself, simplifying the hand off and release processes.



So What's New?

Includes an application plus any dependent binaries or libraries.

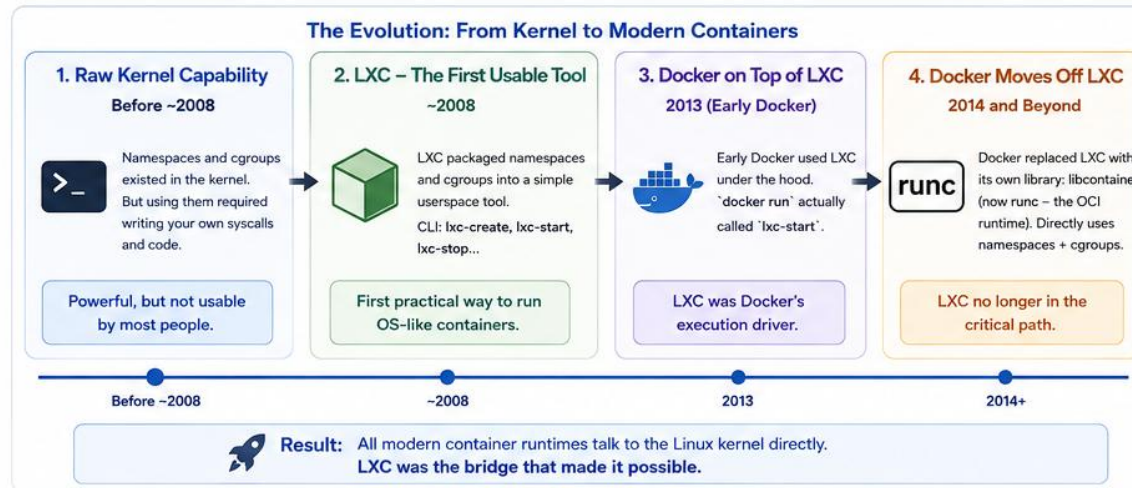
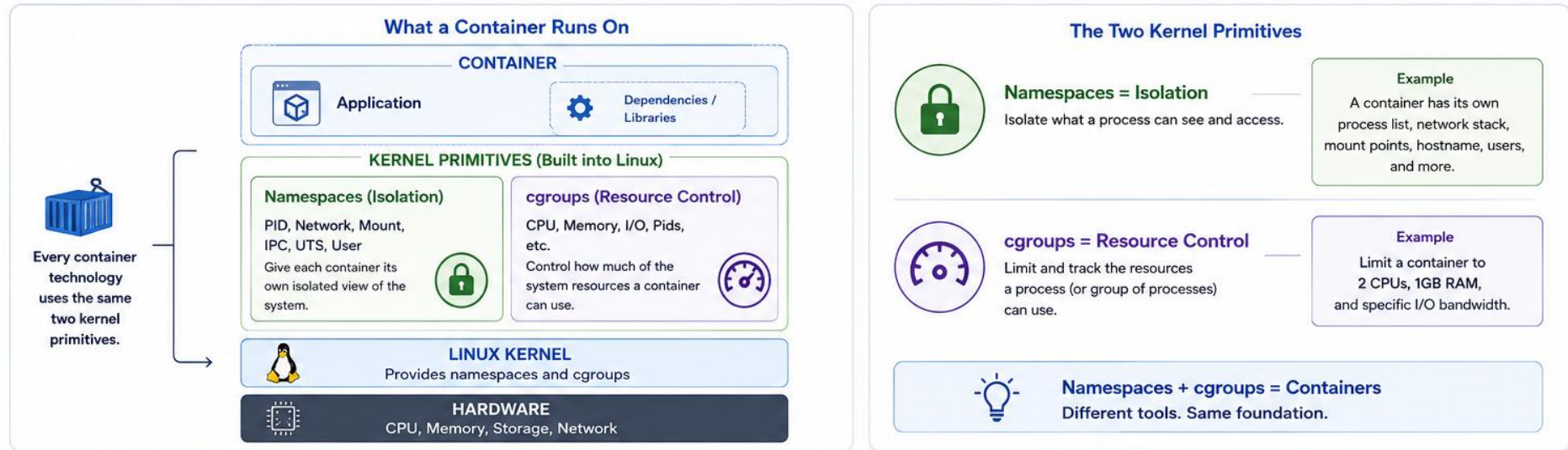
The container runs under the control of the container engine (such as Docker or CRI-O),

The container engine runs on top of the operating system (which can be Windows 10, Windows Server 2016, or Linux depending on the container engine being used).

Container - Introspection

Containers: What They Are Built On – and Why LXC Matters

All modern containers are built on two Linux kernel primitives.



Do Containers Need LXC Today?

✓ **No. Modern containers run with zero LXC involvement.**
Docker (runc), Podman, containerd, CRI runtimes, Kubernetes – all use namespaces & cgroups directly.

Where LXC Still Matters
LXC/LXD (or Incus) continues to exist as its own product, excellent for running full system containers with an OS-like experience.
But it is no longer required by Docker or Kubernetes.

Why Teach LXC?

- ✓ First mainstream tool to make kernel container features usable
- ✓ Teaches the real building blocks (namespaces & cgroups)
- ✓ Historical bridge from raw kernel to modern container ecosystem
- ✓ Helps you understand what modern runtimes are doing under the hood

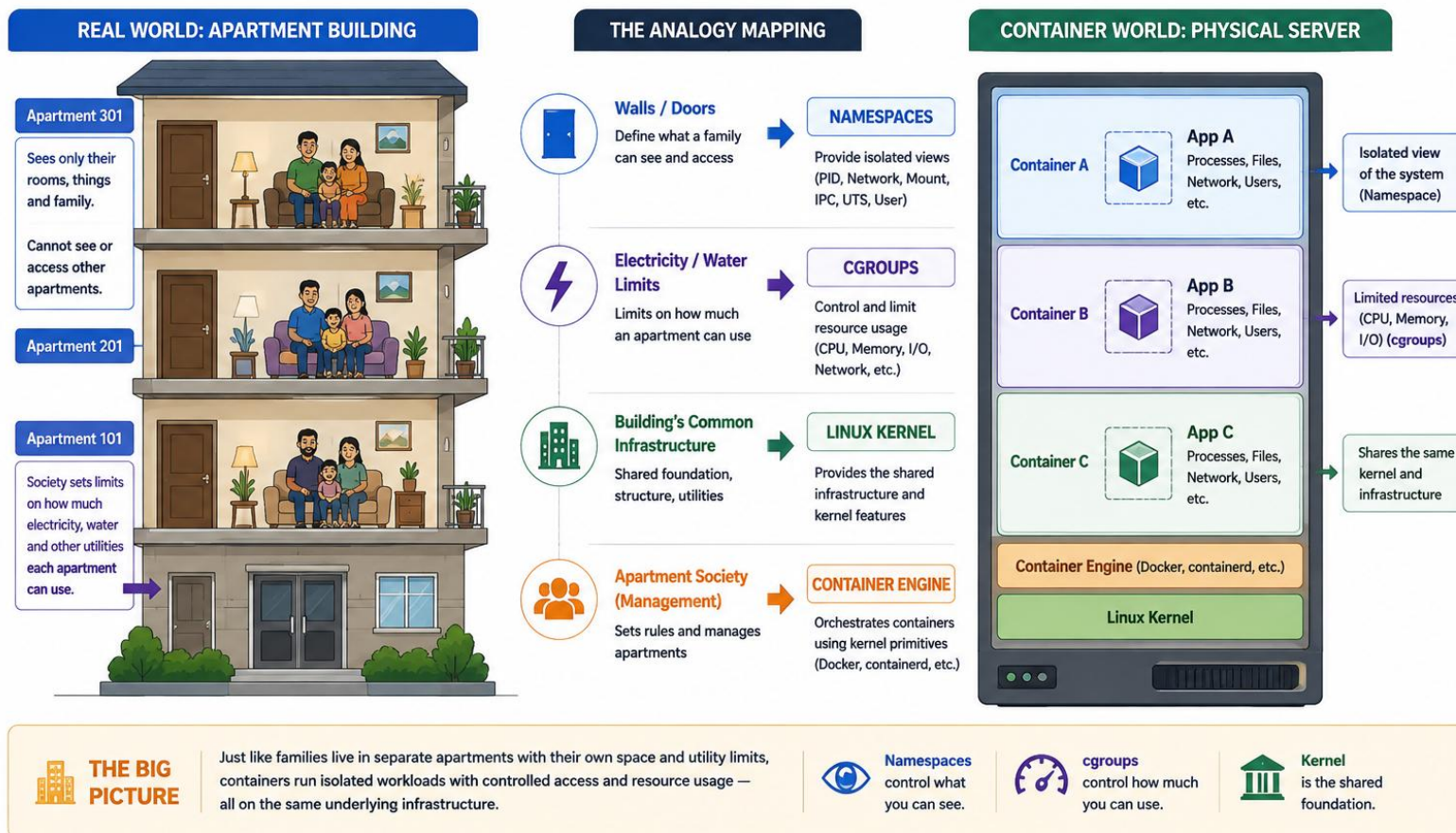


Bottom Line: Containers exist because of Linux kernel features. LXC made them usable. Docker (and others) made them ubiquitous.

Control Groups & Namespaces

Think of a Container as an Apartment

Isolation is about what you can see (**Namespaces**). Control is about how much you can use (**cgroups**).



Control Groups

- **Cgroups (Control Groups)** is a **Linux kernel feature** that limits and tracks how much of the system's actual resources — CPU, memory, disk I/O — a group of processes is allowed to use.
- Every process on Linux descends from one parent (the init process) — so all processes naturally form a tree.
- Cgroups let you group processes together and attach resource limits to that group — e.g., "this group of containers gets at most 2 CPU cores and 4GB RAM, combined."
- Built by Google engineers in 2006 to solve exactly this problem at scale, and merged into the mainline Linux kernel in 2008.
- It's one of the two kernel primitives (alongside namespaces) that every container technology — LXC, Docker, Kubernetes — is built on top of.
- **Without cgroups** one runaway container can eat all the RAM on the host and crash every other container.
- **With cgroups:** that container hits its ceiling and gets throttled everyone else keeps running."
- If one process in the group tries to exceed the limit, the kernel throttles or kills it — the rest of the system is protected.

Control Groups

cgroup: Control Groups provide a mechanism for aggregating/partitioning sets of tasks, and all their future children, into hierarchical groups with specialized behaviour.

• **namespace**: wraps a global system resource in an abstraction that makes it appear to the processes within the namespace that they have their own isolated instance of the global resource.

In short:

- Cgroups = limits how much you can use;
- namespaces = limits what you can see (and therefore use)

Linux cgroups: What Each One Controls

cgroups limit, allocate, and account for system resources used by a group of tasks (processes).

cgroup Subsystem	What It Controls	What It Means	Example / How It Helps	Icon
 blkio (Block I/O)	Limits input/output access to block devices.	Restricts how much a group can read from or write to devices such as disk, SSD, or USB.	 Prevents a container from dominating disk I/O.	
 cpu (CPU)	Controls access to CPU using the scheduler.	Shares CPU time among groups based on configured limits and weights.	 Limits CPU usage or guarantees a fair share.	
 cpuacct (CPU Accounting)	Generates reports on CPU resources used.	Provides CPU usage statistics for tasks in the group.	 Helps monitor and analyze CPU consumption.	
 cpuset (CPU & Memory Affinity)	Assigns specific CPUs and memory nodes.	Binds tasks to specific CPUs (cores) and/or NUMA memory nodes.	 Improves performance by keeping tasks close to assigned resources.	
 devices (Device Access)	Allows or denies access to devices.	Controls which device files (e.g., /dev/sda, /dev/tty) tasks in the group can use.	 Enhances security by restricting device access.	
 freezer (Process Freezing)	Suspends or resumes tasks.	Allows all tasks in a group to be frozen (suspended) and later thawed (resumed).	 Useful for checkpointing or pausing workloads.	
 memory (Memory)	Limits memory usage and reports usage.	Sets limits on how much memory a group can use and reports memory statistics.	 Prevents out-of-memory issues and tracks usage.	
 net_cls (Network Classification)	Tags network packets with a class ID.	Adds a class identifier to packets so traffic control (tc) can identify them.	 Enables monitoring or shaping traffic from specific groups.	
 net_prio (Network Priority)	Sets the priority of network traffic.	Allows setting the priority (class) of traffic per network interface.	 Helps prioritize critical network traffic.	
 ns (Namespace)	The namespace subsystem.	Handles namespace lifecycle for tasks in a cgroup.	 Used by container runtimes to manage namespaces.	
 perf_event (Performance Events)	Identifies membership for performance analysis.	Tracks performance events (like cache misses, cycles) for tasks in the group.	 Useful for profiling and performance troubleshooting.	

 **Key Takeaway:** cgroups control how much of the system's resources a group of processes can use, and provide visibility into their usage.

Namespaces

- Namespaces are a Linux kernel feature that gives a process (or group of processes) its own isolated view of the system — even though it's still running on the same shared kernel as everything else.
- Run `ps aux` inside a container and you'll see maybe 3 processes. Run it on the host and you'll see hundreds — including that same container process, just under a different PID. Same process, two different views — that's a namespace."

Linux Namespaces: What Each One Isolates

Namespaces give a container its own **isolated view** of system resources.

Namespace	What It Isolates	What It Means	Example (Inside Container vs Host)		Icon
PID (Process ID)	Process IDs	A process in a container can be PID 1, while on the host it has a different PID.	Inside Container PID 1	VS On Host PID 12345	
Network (NET)	Network interfaces, IP addresses, routing tables	The container has its own network interfaces, IP addresses, and routing rules.	Inside Container 10.0.0.2	VS On Host 192.168.1.10	
Mount (MNT)	Filesystem mount points	The container sees its own filesystem structure and mount points.	Inside Container / (root)	VS On Host / (root)	
UTS (UNIX Timesharing System)	Hostname	The container can have its own hostname, independent of the host.	Inside Container web-container	VS On Host prod-server	
IPC (Inter-Process Communication)	Inter-process communication resources	The container has its own message queues, semaphores, shared memory, etc.	Inside Container	VS On Host	
User (USER)	User and group IDs	A process can be "root" (UID 0) inside the container but map to an unprivileged user on the host.	Inside Container root (UID 0)	VS On Host UID 1000	

 Namespaces provide isolation of view — each container gets its own world, while sharing the same kernel.

Namespaces and cgroups **solve two different problems** that both look like "isolation" but aren't: **namespaces** stop a **process from seeing other processes'** stuff; **cgroups stop a process from consuming the whole host's resources**.

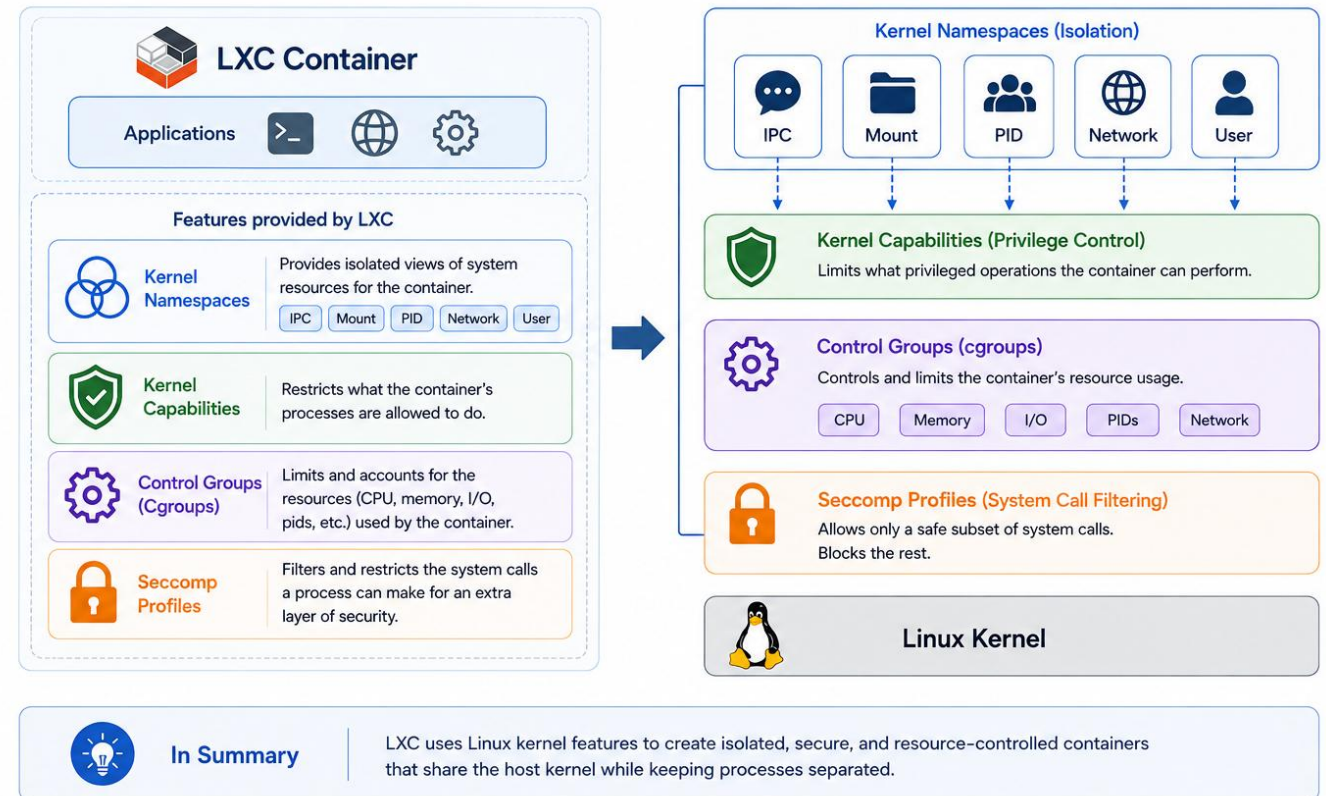
A container needs both — isolation without resource limits means one noisy container can still starve the others, even if it can't see them.

Linux Containers LXC

- LXC is an abbreviation used for Linux Containers which is an operating system that is used for running multiple Linux systems virtually on a controlled host via a single Linux kernel.

LXC – Key Features

LXC provides the building blocks to run secure, isolated system containers.



Linux Containers LXC

- **LXC (Linux Containers)** is a **userspace interface** to the **Linux kernel's containment features**. It is not an operating system.
- It's a set of tools and libraries that let a single Linux kernel run multiple isolated user-space environments, without the overhead of full hardware virtualization.
- How it works — two kernel mechanisms, used together: Namespaces & Cgroups (Control Groups)
- Why it matters:
- All containers on a host share one kernel — no guest OS, no hypervisor, no boot time.
- Each container still behaves like an isolated Linux system to the processes running inside it.
- This is the same underlying kernel machinery — namespaces + cgroups — that every later container technology (Docker, Podman, Kubernetes runtimes) also relies on, even though most of them no longer use LXC's own tooling to drive it.

Linux Containers LXD

LXD is a management layer built on top of LXC.

Technically, it's a REST API that connects to liblxc (the LXC software library).

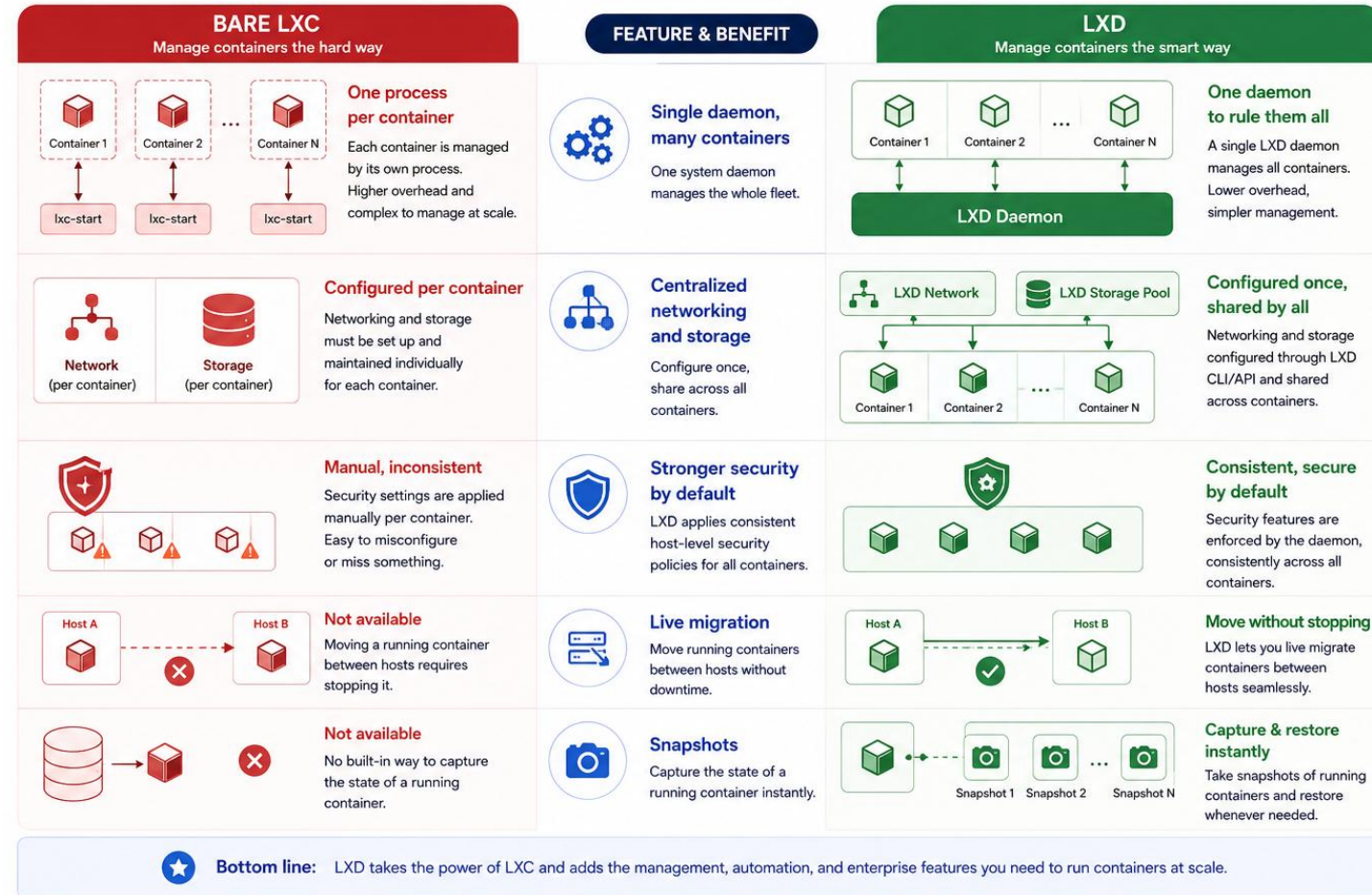
The problem it solves: Bare LXC gives you kernel-level container primitives, but no central way to manage them at scale.

Every container needs its own separate LXC process, there's no single daemon coordinating them, and networking/storage/security have to be configured container-by-container.

That's fine for one or two containers on a workstation; it doesn't hold up once you're running a fleet on a host. LXD exists to turn "a pile of individually-managed containers" into something one daemon can administer centrally.

LXD: The Next Level Over Bare LXC

LXD builds on LXC to deliver enterprise-grade container management.



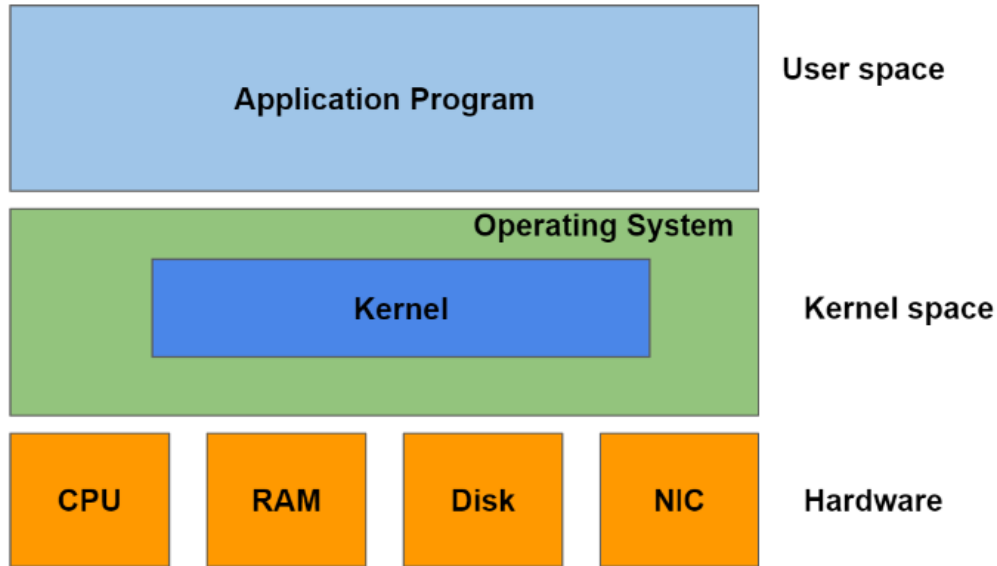
Comparing LXC & LXD

LXC vs LXD – Key Differences at a Glance

LXD builds on LXC to deliver enterprise-grade container management.

 LXC (Bare LXC)	FEATURE / CAPABILITY	 LXD (LXC Daemon)
A virtual environment creation tool built by Google, IBM, etc.		An add-on for LXC that provides advanced features and functionalities.
 Multiple processes are needed for multiple containers and hence it is not flexible.	 Process Management	 LXD makes it flexible by providing a single process for multiple containers.
 Snapshots, Live Migration etc are some of the features which are not supported by LXC.	 Advanced Features	 LXD supports snapshots and lives migration features.
 Scalability functionality is not provided by LXC and hence users shift to other virtual solutions.	 Scalability	 With the use of LXD, scalability is achieved in LXC.
 Management capabilities are poor, especially in the case of network and storage.	 Management Capabilities	 It has better management capabilities like storage pooling.
 It is not user friendly and needs the expertise to handle the processes.	 User Experience	 It provides a user-friendly interface.
 After data processing, the data cannot be retrieved.	 Data Retrieval	 Data retrieval functionality after data processing is provided in LXD.
 C API is used by the LXC.	 API	 LXD uses REST API.
 Bottom Line: LXD takes the power of LXC and adds centralized management, advanced features, better scalability, and a user-friendly experience for running containers at scale.		

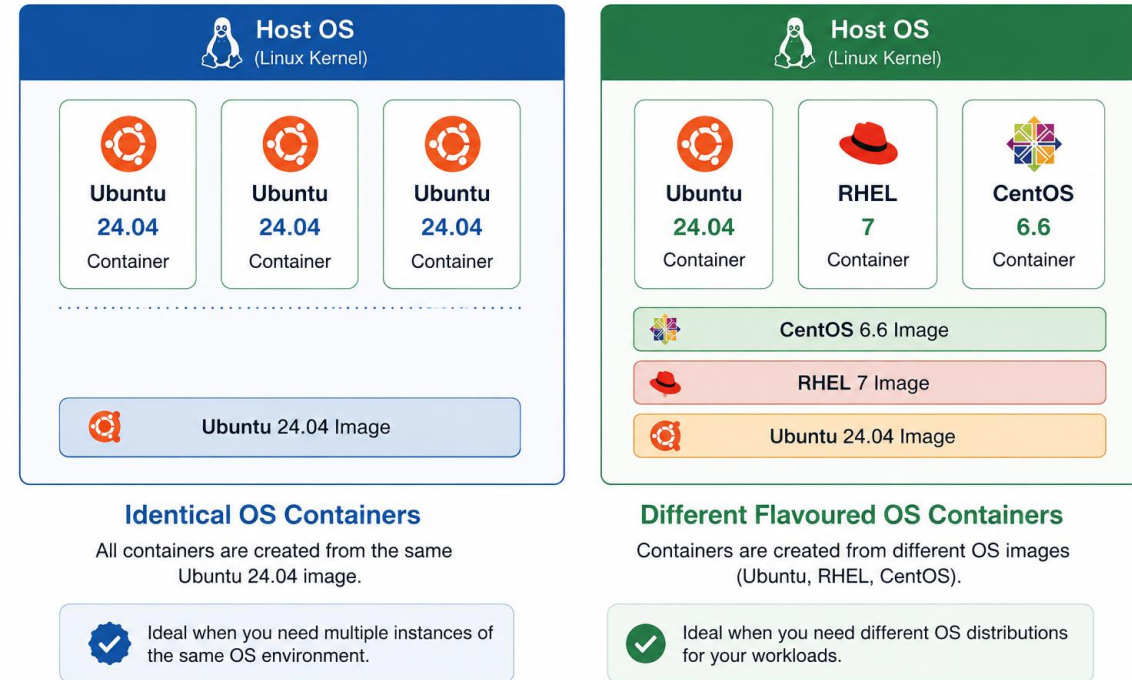
Container Types – OS Containers



Working of an Operating System

- **OS containers** are virtual environments that **share the kernel** of the **host operating system** but provide **user space isolation**.
- For all practical purposes, you can think of **OS containers as VMs**. You can install, configure and run different applications, libraries, etc., just as you would on any OS.
- Just as a VM, **anything running inside a container** can only see **resources that have been assigned to that container**.

Types of OS Containers



- OS containers are useful when you want to run a fleet of identical or different flavors of distros.

Container Types – Application Containers

APPLICATION CONTAINERS

Lightweight. Portable. Built for Modern Applications.



Designed to package and run a single service

Each container wraps one service or process and its dependencies.



All containers share host kernel

Containers use the host OS kernel, making them lightweight and fast.



Subtle differences from operating system containers

Application containers focus on processes and services, not full operating system environments.



Examples: Docker, Rocket

Popular application container technologies.



Docker: runs a single process on creation

Each Docker container starts one main process.



OS containers: run many OS services, for an entire OS

Used for full system environments with multiple services.



Create application containers for each component of an app

Break applications into independent, containerized components.



Supports a micro-services architecture

Ideal for building, deploying, and managing microservices.



DevOps: developers can package their own components in application containers

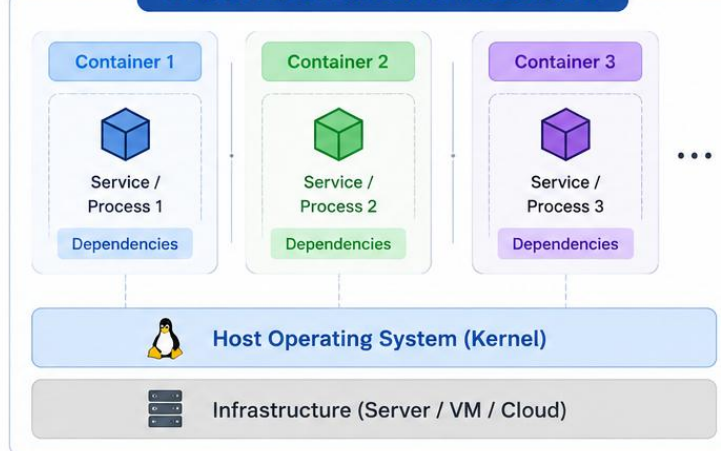
Enables consistent delivery across environments.



Supports horizontal and vertical scaling

Scale out with more containers or scale up resources as needed.

HOW APPLICATION CONTAINERS WORK



KEY TAKEAWAY



Application containers are lightweight, share the host kernel, and are designed to run a single process or service. They power modern DevOps and microservices with speed, portability, and scalability.

AT A GLANCE



Lightweight
& Fast



Portable
Across Environments



Built for
Microservices



Easily
Scalable



DevOps
Friendly

- While OS containers are designed to run multiple processes and services, application containers are designed to package and run a single service.
- Container technologies like Docker and Rocket are examples of application containers.
- So even though they share the same kernel of the host there are subtle differences make them different, which I would like to talk about using the example of a Docker container:

Comparison: OS Containers vs Application Containers

Two approaches to containerization for different needs

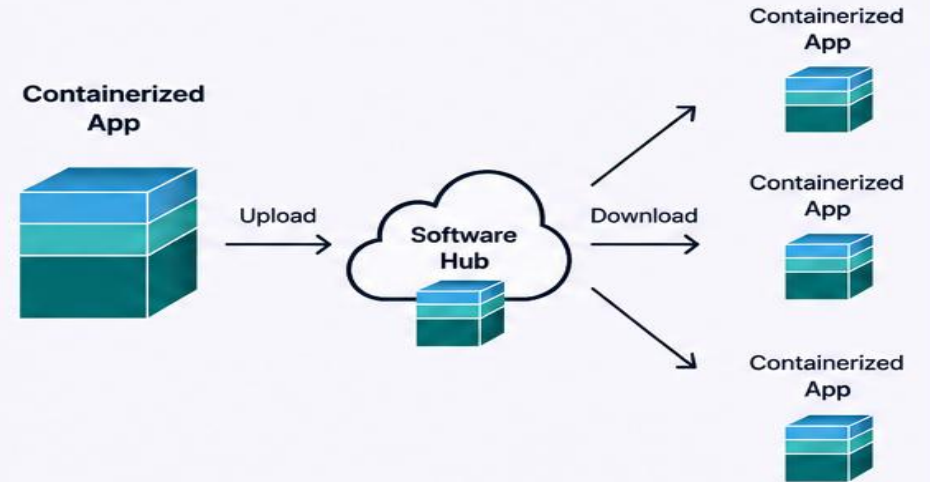
OS CONTAINERS



APPLICATION CONTAINERS



APPLICATION CONTAINER WORKFLOW



- Meant to be used as an OS – run multiple services
- No layered filesystems by default
- Built on cgroups, namespaces, native process resource isolation
- Examples – LXC, OpenVZ, Linux VServer, BSD Jails, Solaris Zones



- Meant to run for a single service
- Layered filesystems
- Built on top of OS container technologies
- Examples – Docker, Rocket

AT A GLANCE



Purpose

OS containers run multiple services (full OS-like environment)

App containers: run a single service or process



File System

OS containers: No layered filesystems by default

App containers: Layered filesystems



Isolation

OS containers: OS-level, native isolation

App containers: Process-level isolation (shares host kernel)



Examples

OS containers: LXC, OpenVZ, Linux VServer, BSD Jails, Solaris Zones

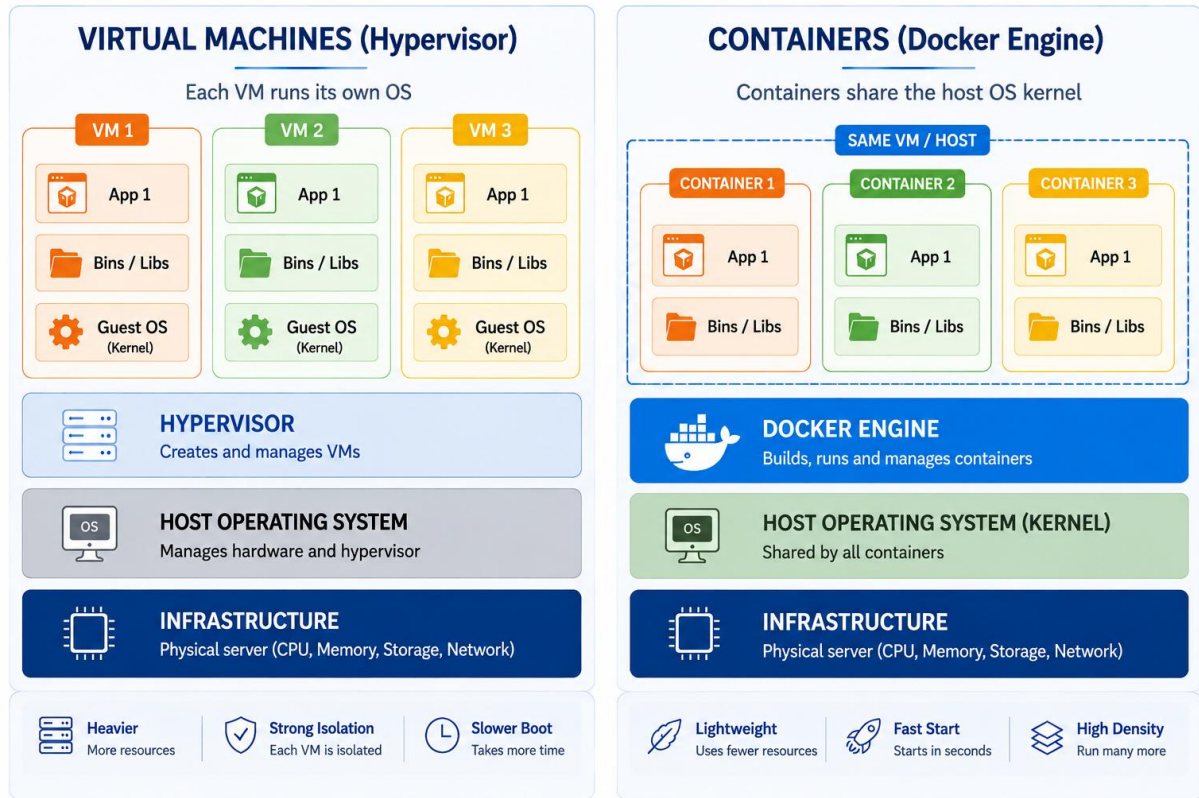
App containers: Docker, Rocket



Distribution

App containers are built once, uploaded to a hub, and distributed for easy download and deployment

VM vs Container



Virtual Machines vs Containers

Two ways to run workloads – very different under the hood

Aspect	Virtual Machine	VS	Container
What it virtualizes	Hardware — each VM gets its own kernel and full OS		The OS — containers share the host kernel
Isolation boundary	Hypervisor (strong, hardware-level)		Namespaces + cgroups (process-level, weaker than a hypervisor)
Startup time	Seconds to minutes (booting a full OS)		Milliseconds to seconds
Typical size	GBs (full guest OS + app)		MBs to low GBs (app + dependencies only)
Density per host	Lower Each VM's guest OS consumes RAM/CPU overhead		Higher No duplicate OS overhead per instance
Kernel	Guest OS can differ from host (e.g., Windows guest on Linux host)		Must share the host's kernel Linux containers need a Linux kernel underneath
Managed by	Hypervisor (e.g., ESXi, KVM, Hyper-V)		Container engine (e.g., Docker) + kernel primitives directly



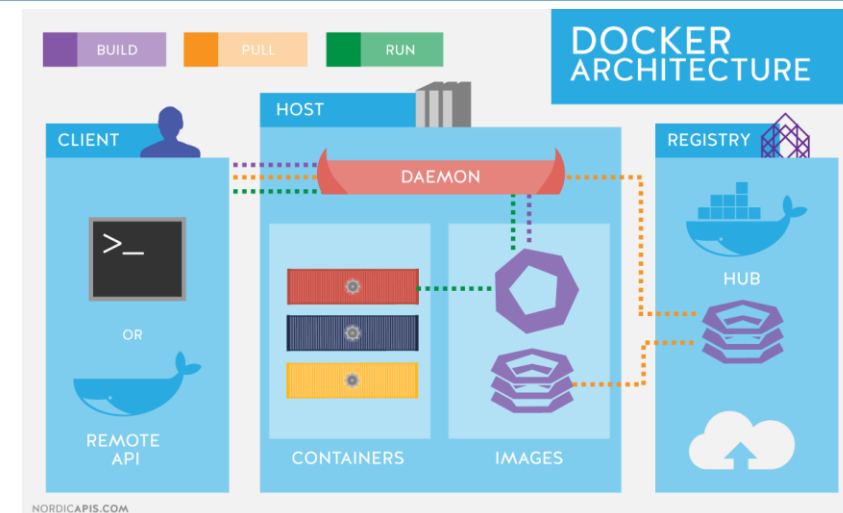
Key Takeaway: VMs virtualize hardware to run OSs. Containers virtualize the OS to run processes.



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Dockers



Dockers - Motivation

1 PROBLEMS IN THE PAST

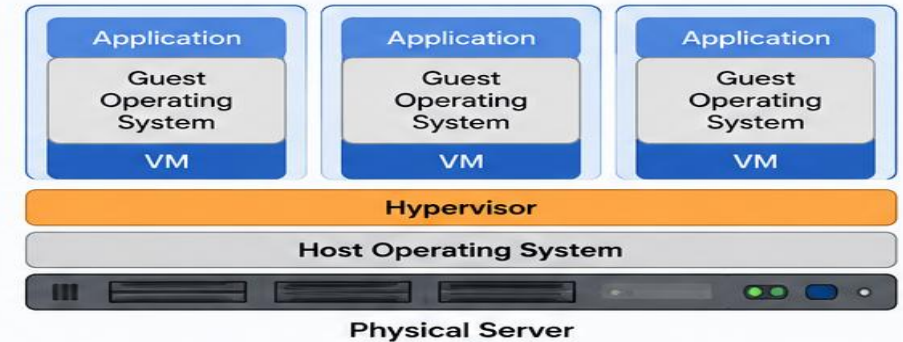
- ⚠ Slow deployment times
- ⚠ Huge costs
- ⚠ Wasted resources
- ⚠ Difficult to scale
- ⚠ Difficult to migrate
- ⚠ Vendor lock in



Virtualization

2 VIRTUALIZATION (HYPERVISOR-BASED)

- One physical server can contain multiple applications
- Each application runs in a virtual machine (VM)



4 CONTAINERS: A LIGHTWEIGHT ALTERNATIVE



Containers

3 LIMITATIONS OF VMS

- Each VM still requires
 - CPU allocation
 - Storage
 - RAM
 - An entire guest operating system
- The more VM's you run, the more resources you need
- Guest OS means wasted resources
- Application portability not guaranteed



KEY TAKEAWAY

Virtualization solved many infrastructure challenges by enabling multiple workloads on one server, but VMs come with overhead and inefficiencies. Containers provide a **lightweight, efficient, and portable** alternative.

Dockers - Introduction

What Is Docker?



Docker is a platform that uses container technology to make it easy to **build, ship, and run applications anywhere**.



Lightweight, open, secure platform

Built with security in mind and designed to be lightweight and open for everyone.



Simplify building, shipping, running apps

Streamlines the entire application lifecycle – from development to deployment.



Runs natively on Linux or Windows Server

Deliver high performance and efficiency on enterprise environments.



Runs on Windows or Mac Development machines (with a virtual machine)

Docker Desktop provides a consistent experience across Windows, Mac, and Linux.



Relies on “images” and “containers”

Images are lightweight, standalone, executable packages; containers are running instances of images.



Docker is a tool designed to make it easier to create, deploy, and run applications by using containers. At a high level, Docker is a utility that can efficiently create, ship, and run containers.

Docker containers wrap a piece of software in a complete file system that contains everything needed to run: **code, runtime, system tools, system libraries**

Docker enables you to quickly, reliably, and consistently deploy applications regardless of environment.

Dockers enables you to **achieve compute density** by running several **isolated containers** on the **same hardware**.

Namespaces provides the isolation

Dockers - Introduction

What is Docker?



Docker is a platform which packages an application and all its dependencies together in the form of containers. This containerization aspect ensures that the application works in any environment.



As you can see in the diagram, each and every application runs on separate containers and has its own set of **dependencies & libraries**. This makes sure that each application is independent of other applications, giving developers surety that they can build applications that will not interfere with one another.



So a **developer** can build a container having different applications installed on it and give it to the **QA team**. Then the QA team would only need to run the container to replicate the developer's environment.

Key Benefits



Consistent Environments



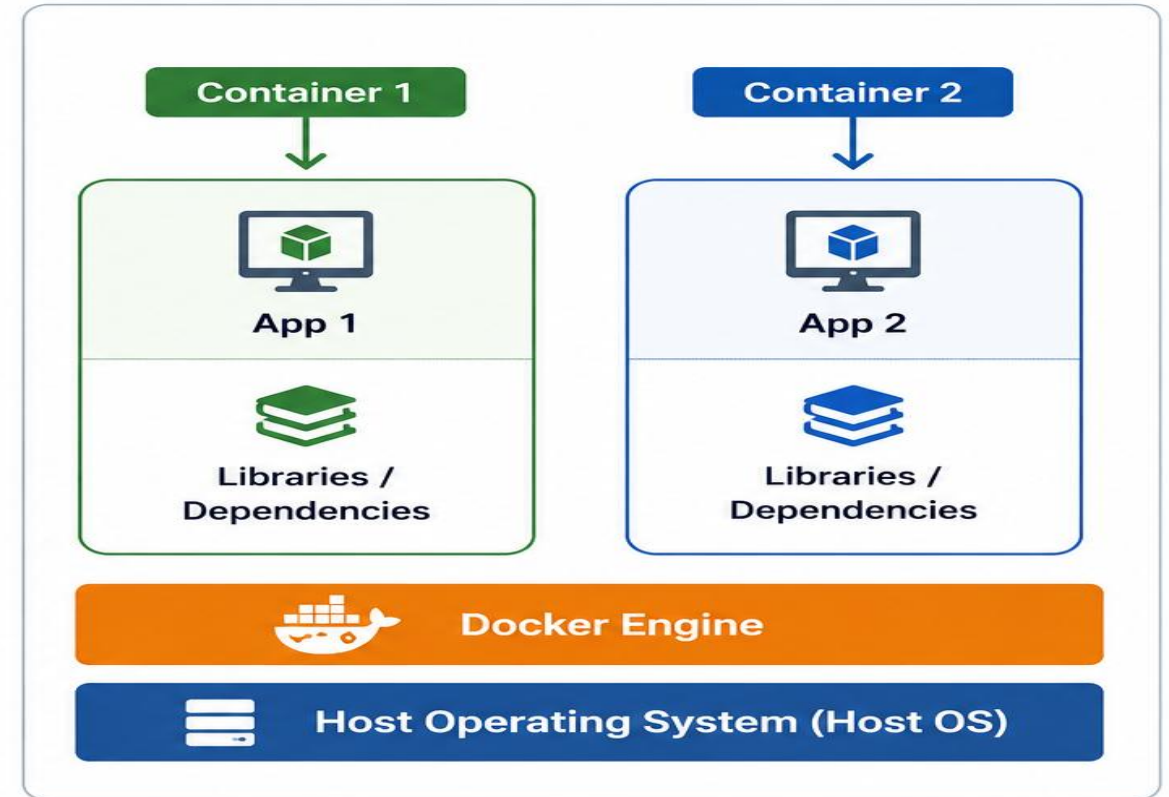
Faster Delivery



Isolation & Security



Portability Anywhere



Docker containers package the application and all dependencies together, ensuring consistency across development, testing, and production environments.

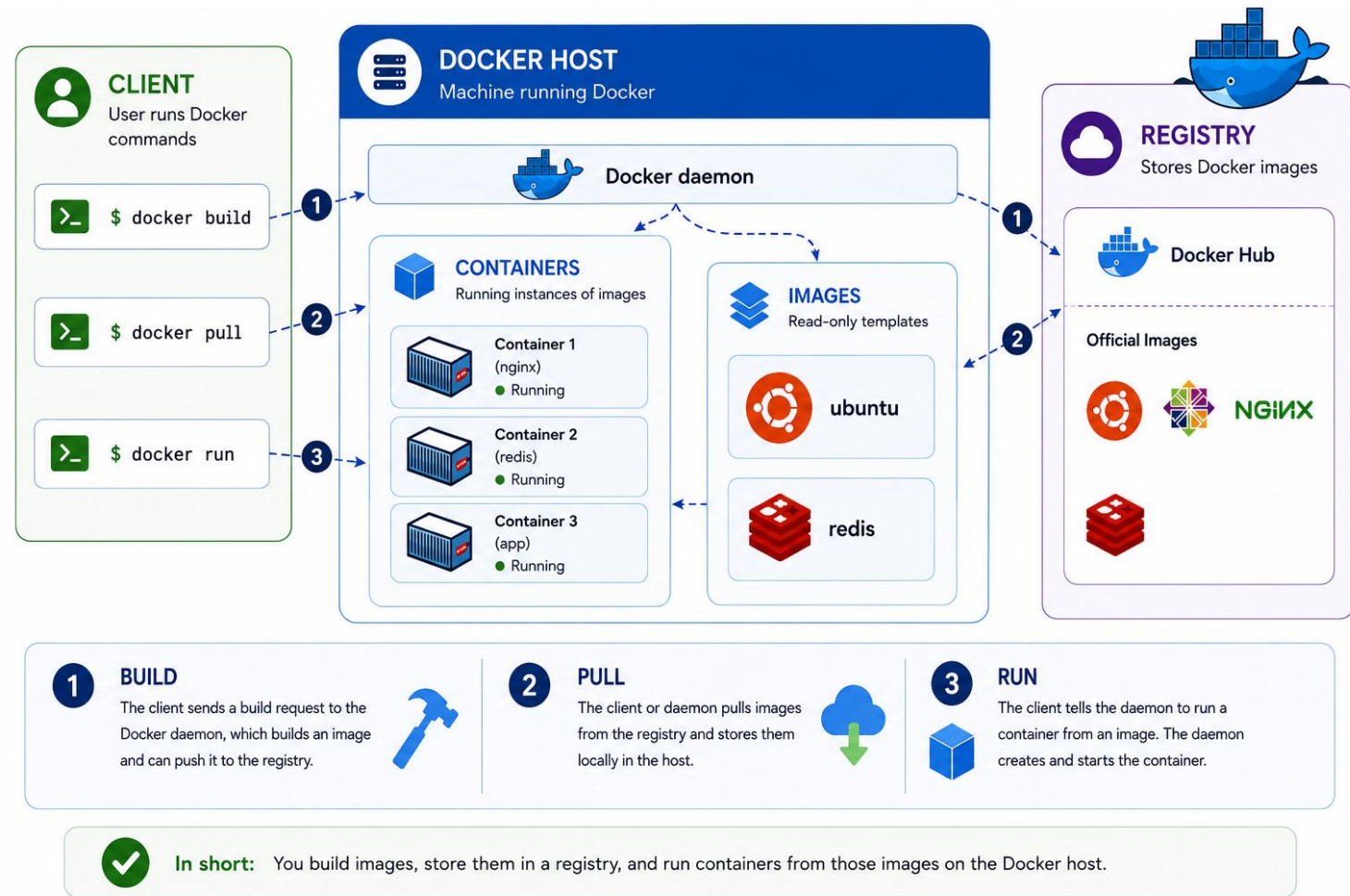
Dockers - Architecture

Docker uses a client-server architecture.

The Docker *client* talks to the Docker *daemon*, which does the heavy lifting of building, running, and distributing your Docker containers.

The Docker client and daemon *can* run on the same system, or you can connect a Docker client to a remote Docker daemon. The Docker client and daemon communicate using a REST API, over UNIX sockets or a network interface.

Another Docker client is Docker Compose, that lets you work with applications consisting of a set of containers. Docker compose is a built in CLI plugin



"The daemon delegates actual container creation to containerd → runc (OCI runtime)"

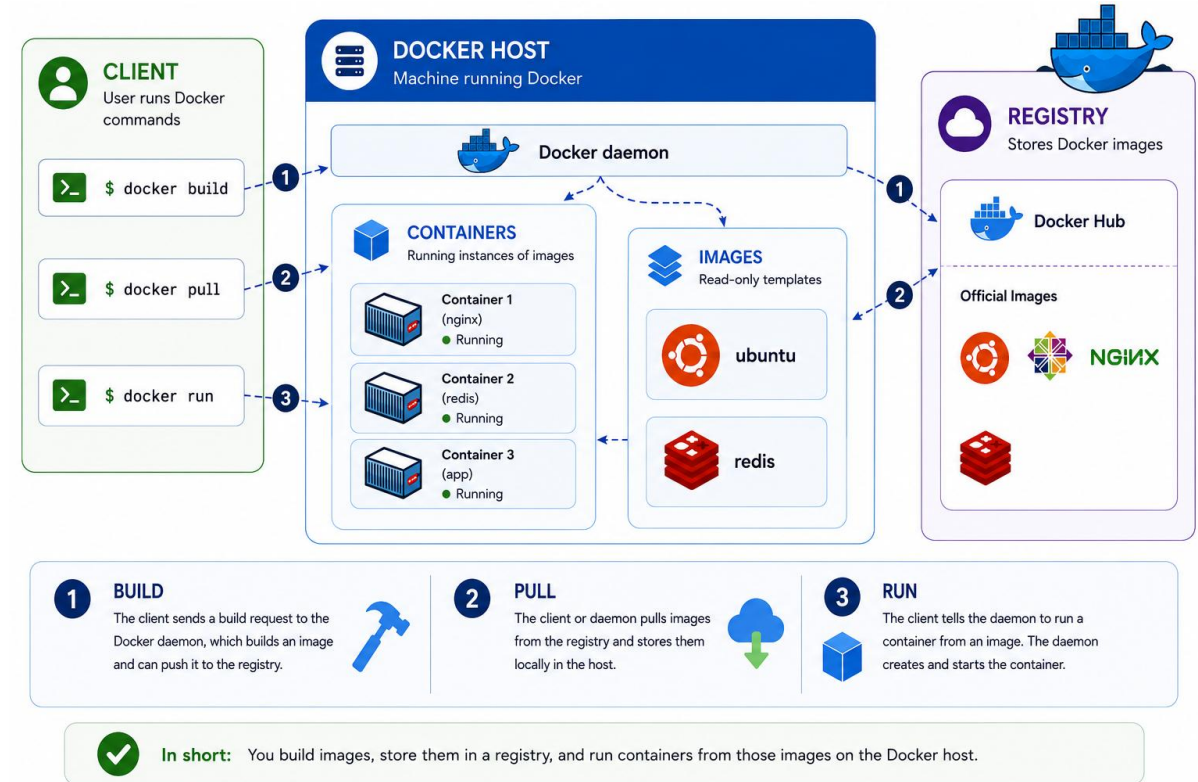
Dockers – Components - Daemon

The Docker daemon, also known as 'dockerd', consistently listens to the requests put forward by the Docker API.

It is used to carry out all the heavy tasks such as creating and managing Docker objects including containers, volumes, images, and networks.

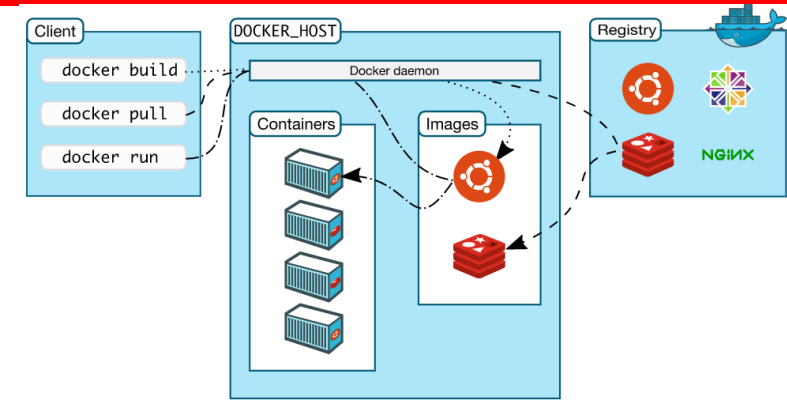
A Docker daemon is also capable of communicating with other daemons in the same or different host machines.

For example, in the case of a swarm cluster, the host machine's daemon can communicate with daemons on other nodes to carry out tasks.



Dockers – Components – Client

The **Docker client** (docker) is the primary way that many Docker users interact with Docker. The Docker users can leverage simple HTTP clients like Command line to interact with Docker. When a user executes a Docker command such as Docker run, the CLI will send this request to the dockerd via the REST API. The Docker CLI can also communicate with over one daemon.



Docker Desktop

Docker Desktop is an easy-to-install application for your Mac , Windows , Linux environment that enables you to build and share containerized applications and microservices. Docker Desktop includes the Docker daemon (`dockerd`), the Docker client (`docker`), Docker Compose, Docker Content Trust, Kubernetes, and Credential Helper. For more information, see [Docker Desktop](#).

Docker registries

A Docker *registry* stores Docker images. Docker Hub is a public registry that anyone can use, and Docker is configured to look for images on Docker Hub by default. You can even run your own private registry.

When you use the `docker pull` or `docker run` commands, the required images are pulled from your configured registry. When you use the `docker push` command, your image is pushed to your configured registry.

Dockers – Components – Registries

Docker registries

A Docker *registry* stores Docker images. Docker Hub is a public registry that anyone can use, and Docker is configured to look for images on Docker Hub by default. You can even run your own private registry.

When you use the `docker pull` or `docker run` commands, the required images are pulled from your configured registry. When you use the `docker push` command, your image is pushed to your configured registry.

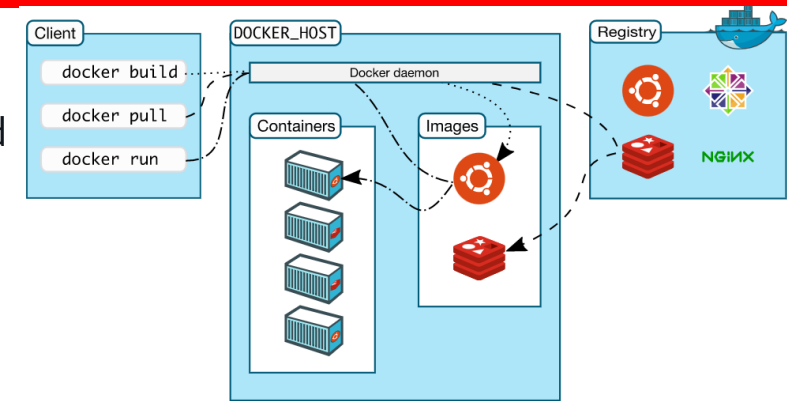
The official [Docker registry](https://hub.docker.com/) called Dockerhub contains several official image repositories.

A repository contains a set of similar Docker images that are uniquely identified by Docker tags.

Dockerhub provides tons of useful official and vendor-specific images to its users.

Some of them include Nginx, Apache, Python, Java, Mongo, Node, MySQL, Ubuntu, Fedora, Centos, etc.

You can even create your private repository inside Dockerhub and store your custom Docker images using the Docker push command.



Docker Objects

A Docker user frequently interacts with Docker objects such as:



Images

Read-only templates used to create containers.



Containers

Runnable instances of images – isolated processes.



Volumes

Persistent data storage for containers.



Plugins

Extend Docker's functionality.



Networks

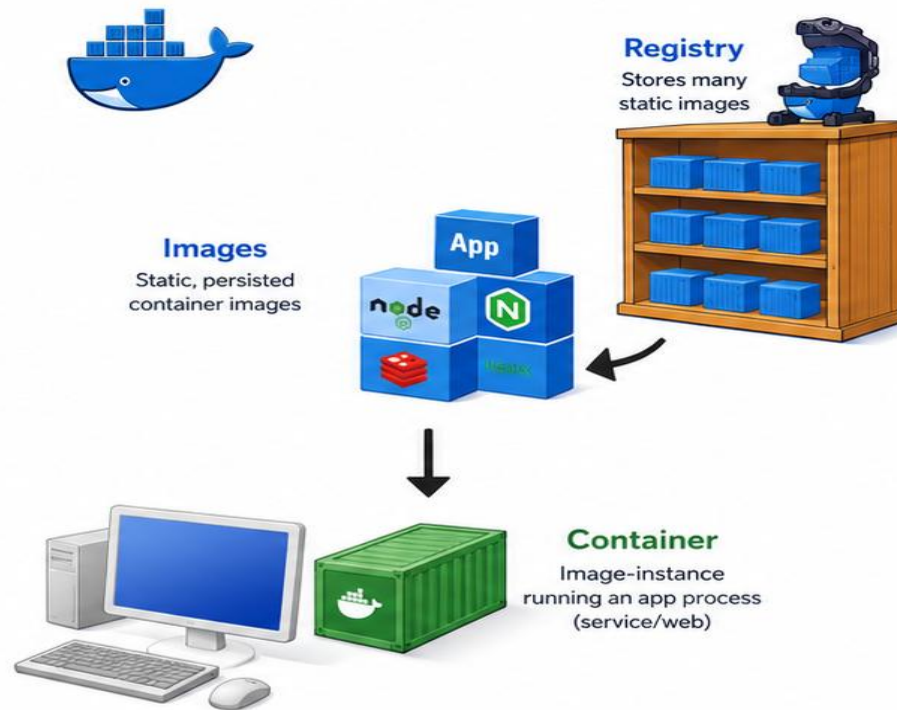
Enable communication between containers.



and so on.

Docker also includes secrets, configs, services and more.

Basic taxonomy in Docker



Docker Registries



Hosted Docker Registry

Docker Trusted Registry on-prem.



Docker Hub Registry

Docker Trusted Registry on-cloud



Azure Container Registry

AWS Container Registry



AWS Container Registry

Amazon Elastic Container Registry



Google Container Registry

Google Container Registry



Quay Registry

Quay Container Registry



Other Cloud

Other Cloud Registries

On-premises
(‘n’ private organizations)

Public Cloud
(specific vendors)



How it works:

1 Build

You build an image from a Dockerfile using docker build.



2 Push / Pull

Images are pushed to a registry or pulled from it (docker push / pull).



3 Run

You run a container from the image (docker run).



Outcome:

Consistent, portable applications across any environment.



Docker Objects - Images

Docker Images are read-only templates that are built using multi-layers of file.

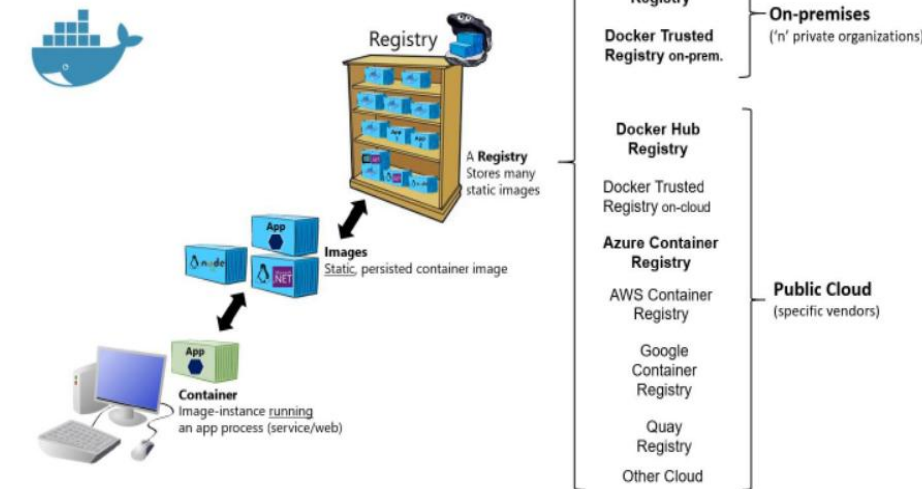
You can build Docker images using a simple text file called Dockerfile which contains instructions to build Docker images.

The first instruction is a FROM instruction which can pull a base image from any Docker registry. Once this base image layer is created, several instructions are then used to create the container environment. Each instruction adds a new layer on top of the previous one.

A Docker image is simply a **blueprint of the container environment**. Once you create a container, it creates a writable layer on top of the image, and then, you can make changes. The images all the metadata that describes the container environment. You can either directly pull a Docker image from Dockerhub or create your customized image over a base image using a Dockerfile.

Once you have created a Docker image, you can push it on **Dockerhub** or any other registry and share it with the outside world.

Basic taxonomy in Docker

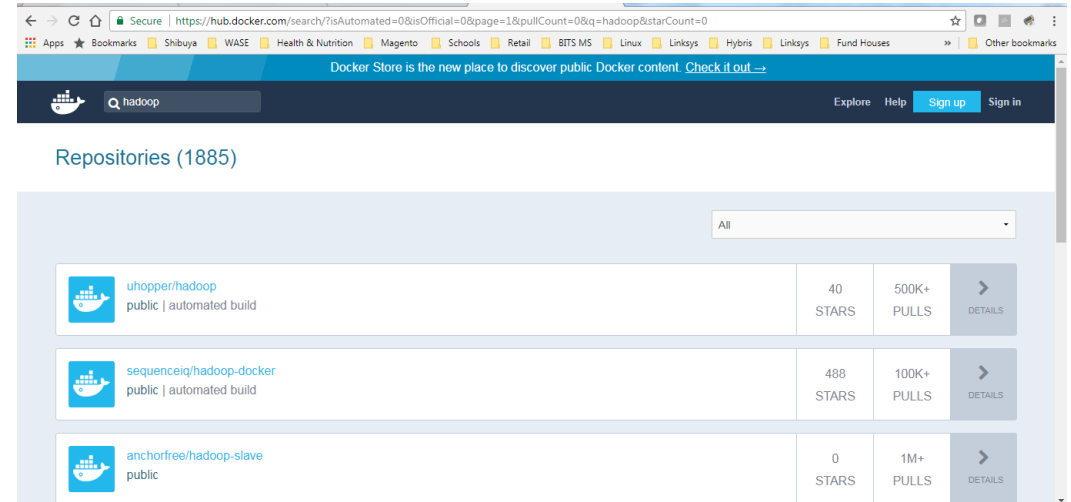


Docker Objects - Images

docker image index: a repository (public or private) for docker images

- Docker images constitute the base of docker containers *from* which everything starts to *form*. They are very similar to default operating-system disk images which are used to run applications on servers or desktop computers.
- Having these images (e.g. Ubuntu base) allow seamless portability across systems. They make a solid, consistent and dependable base with everything that is needed to run the applications. When everything is self-contained and the risk of system-level updates or modifications are eliminated, the container becomes immune to external exposures which could put it out of order - *preventing the dependency hell*.

Note: Repositories contain images from un verified sources. Exercise caution. Always use the official images

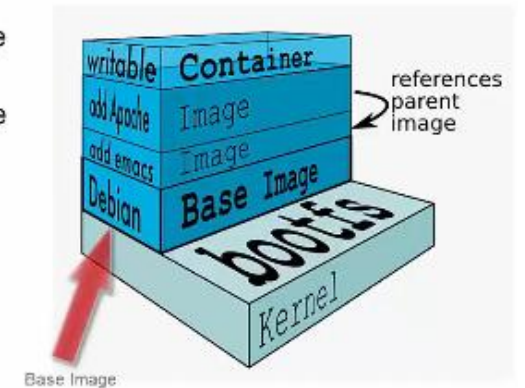


Docker Objects - Images

- As more layers (tools, applications etc.) are added on top of the base, new images can be formed by *committing* these changes. When a new container gets created from a saved (i.e. committed) image, things continue from where they left off. And the [union file system](#), brings all the layers together as a single entity when you work with a container.
- These base images can be explicitly stated when working with the **docker CLI** to directly create a new container or they might be specified inside a **Dockerfile** for automated image building.

Image Layers

- Images are comprised of multiple layers
- A layer is also just another image
- Every image contains a base layer
- Docker uses a copy on write system
- Layers are read only



Note: Repositories contain images from unverified sources. Exercise caution. Always use the official images

Docker Objects - Images

The Container Writable Layer

Docker uses a layered filesystem. A read-only image is combined with a thin writable layer to create a container.



Docker creates a top writable layer for containers

This is the container layer where all changes (e.g., file updates, new files, deletions) are stored.



Parent images are read only

The image layers (e.g., BusyBox, Debian, etc.) are immutable and shared across containers.



All changes are made at the writable layer

Changes in the container do not affect the underlying image layers.

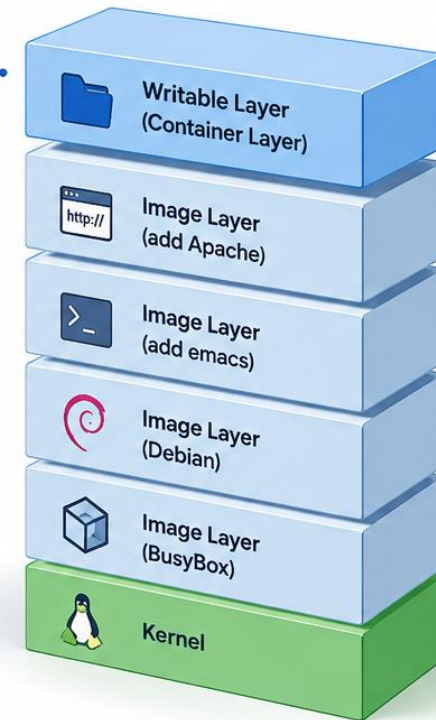


In short

Docker containers share image layers (read-only) and add a thin writable layer on top. This makes containers lightweight, efficient, and easy to manage.

Writable layer
All writes, updates, deletes happen here.

How the writable layer works



Read-only image layers

These layers are immutable and shared.

Host Kernel

Shared by the host and all containers.

Writable Layer
(Container Layer)

Read-only Image
Layers

Host Kernel
(Shared)

Docker Objects - Containers

Docker containers are isolated, encapsulated, packaged, and secured application environments that contain all the packages, libraries, and dependencies required to run an application.

For example, if you create a container associated with the Ubuntu image, you will have access to an isolated Ubuntu environment. You can also access the bash of this Ubuntu environment and execute commands.

Containers have all the access to the resources that you define while using the **Dockerfile** while creating an image. Such configurations include *build context, network connections, storage, CPU, memory, ports, etc.*

For example, if you want access to a container with libraries of Java installed, you can use the Java image from the Dockerhub and run a container associated with this image using the Docker run command.

You can also create containers associated with the custom images that you create for your application using the Dockerfiles. Containers are very light and can be spun within a matter of seconds.

Creating a Container

- Use **docker run** command
- Syntax
`sudo docker run [options] [image] [command] [args]`
- Image is specified with `repository:tag`

Examples

```
docker run ubuntu:14.04 echo "Hello World"
docker run ubuntu ps ax
```

Docker Objects - DockerFiles

1. Reproducibility.

- A **Dockerfile** is a **text-based, version-controllable recipe**.
- Anyone with the same Dockerfile and the same build context produces the same image, byte-for-byte layer structure, every time versus manually running commands inside a container and committing it (docker commit), which is undocumented, unrepeatable, and leaves no record of *how* the image got that way.

2. It's the only way to legitimately customize a base image.

- Everything deployment requires a official base image & your schema, your code, your dependencies. There's no docker run flag that adds a file into an image; that's specifically what COPY and RUN in a Dockerfile do.

3. Documentation as a side effect.

- A Dockerfile is self-documenting — anyone reading it sees exactly what's installed, what ports are expected, what user it runs as. This is why the "COPY requirements.txt before COPY code" ordering matters pedagogically beyond just caching — a well-written Dockerfile tells a reader the dependency graph of the build just by its instruction order.

Docker Objects - DockerFiles

4. Automation and CI/CD.

- Docker build from a Dockerfile is a single deterministic command a pipeline can run unattended.

5. Layered efficiency you don't get any other way.

- Each instruction becomes a cached, reusable layer.
- Rebuilding after a small code change reuses every unchanged layer instead of redoing the whole image .
- This is the entire reason the dependency-before-code ordering rule exists.

Naming convention

- **Default filename is exactly Dockerfile** capital D, no file extension. docker build looks for that literal filename in the build context; nothing else works without extra flags. It's better to adopt this early on even when working on case-insensitive OS.
- **Custom names require the -f flag:** docker build -f Dockerfile.prod . A common real-world pattern is naming variants for different environments or stages Dockerfile.dev, Dockerfile.prod, Dockerfile.test each with a different base image or build steps, selected explicitly at build time.
- **Location matters as much as name.** By default Docker looks in the root of the build context (the directory you pass as the final argument to docker build).



Dockerfile Instruction Order

Build fast. Cache smart. Run secure.

RULE-BASED ORDER

Docker enforces or strongly expects this

1		FROM	Must always be the first instruction (aside from optional ARG before it). Declares the base image everything else builds on.	<code>FROM python:3.11-slim</code>
2		LABEL / ARG	Metadata and build-time variables, early since later instructions may reference them.	<code>ARG APP_VERSION=1.0.0</code> <code>LABEL maintainer="dev@example.com"</code>
3		WORKDIR	Set the working directory before copying files or running commands, so relative paths behave predictably.	<code>WORKDIR /app</code>
4		ENV	Environment variables needed during the build (e.g. DEBIAN_FRONTEND=noninteractive).	<code>ENV DEBIAN_FRONTEND=noninteractive</code> <code>ENV PYTHONDONTWRITEBYTECODE=1</code>

CACHING-DRIVEN ORDER

This is convention, not syntax — but getting it wrong silently makes every rebuild slow

5		COPY dependency manifests first	requirements.txt, package.json, pom.xml — before the rest of the code.	<code>COPY requirements.txt .</code> <code># or</code> <code>COPY package.json package-lock.json ./</code> <code>COPY pom.xml .</code>
6		Install dependencies	RUN pip install, RUN npm ci, etc. — this layer only rebuilds when the manifest changes.	<code>RUN pip install --no-cache-dir -r requirements.txt</code> <code># or</code> <code>RUN npm ci --only=production</code> <code># or</code> <code>RUN mvn -B dependency:go-offline</code>
7		Copy the rest of the application code	<code>COPY . .</code> — this changes on every commit, so it goes last among the “copy” steps deliberately.	<code>COPY . .</code>
8		EXPOSE	Documents the port (informational only, doesn't publish anything).	<code>EXPOSE 8080</code>
9		USER	Switch to a non-root user before the final run step.	<code>USER appuser</code>
10		CMD or ENTRYPOINT	Always last; defines what runs when the container starts.	<code>CMD ["python", "app.py"]</code> <code># or</code> <code>ENTRYPOINT ["unicorn", "app:app"]</code>



Key takeaways

- ✓ Put stable, rarely-changing steps first (manifests + installs) to leverage cache.
- ✓ Put frequently-changing steps last (code copy).
- ✓ Keep the container small, secure, and predictable.

Dockerfile Best Practices

✓ DO'S		
Do	Why	Example
 Use exec-form CMD	Shell-form doesn't forward signals — <code>docker stop</code> hangs instead of shutting down cleanly.	<code>CMD ["python3", "app.py"]</code>
 Clean up package caches in the same RUN layer that installs them	A later layer can't shrink an earlier one — the cache is already baked in permanently.	<code>RUN apt-get update && apt-get install -y curl \</code> <code>&& rm -rf /var/lib/apt/lists/*</code>
 Copy dependency manifests before application code	Preserves layer caching — code edits don't force a full dependency reinstall.	<code>COPY requirements.txt .</code> <code>RUN pip install -r requirements.txt</code> <code>COPY . .</code>
 Exclude unnecessary files from the build context	Smaller, faster builds; keeps secrets and local config out of the image entirely.	<code>dockerignore</code> containing: <code>.git</code> <code>node_modules</code> <code>.env</code>
 Tag images meaningfully	Reproducible deployments, easy rollback to a known version.	<code>docker build -t myapp:1.2.0 .</code>

✗ DON'TS		
Don't	Why	Example (wrong)
 Don't run <code>apt-get install</code> without <code>apt-get update</code> first	Stale/empty package index — the exact failure walked through in the broken-Dockerfile exercise.	<code>RUN apt-get install -y curl</code>
 Don't use <code>FROM ubuntu:latest</code> (or any <code>:latest</code>)	Image contents silently change between builds — not reproducible.	<code>FROM ubuntu:latest</code>
 Don't <code>COPY . .</code> before installing dependencies	Destroys layer caching — every code change forces a full dependency reinstall.	<code>COPY . .</code> <code>RUN pip install -r requirements.txt</code>
 Don't leave the container running as <code>root</code>	Default behavior if <code>USER</code> is never set — unnecessary privilege inside the container.	(no <code>USER</code> instruction present anywhere in the Dockerfile — root by default, silently)
 Don't hardcode secrets into the <code>Dockerfile</code>	Permanently baked into the image layer history — retrievable even if "removed" in a later layer.	<code>ENV DB_PASSWORD=SuperSecret123</code>

Docker Image Lifecycle

Build Once. Share Anywhere. Run Everywhere.

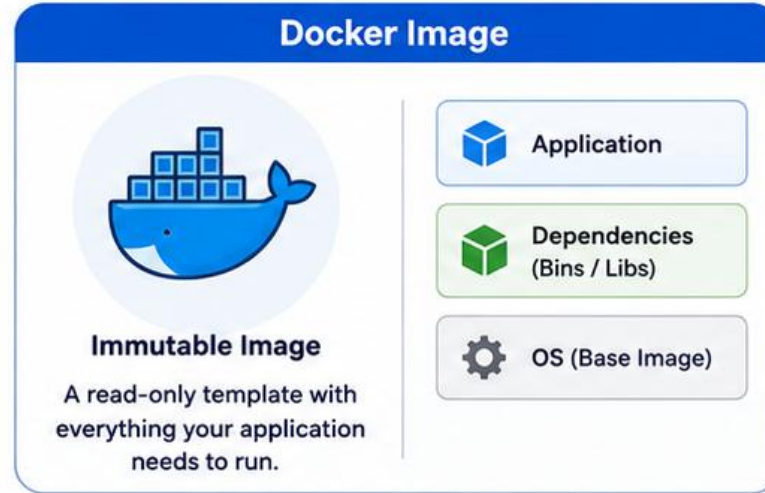
1 BUILD

Auto-build Docker image

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

Dockerfile defines the steps to build the image



Container (Running)

Image becomes a running container



✓ Everything you need to **run** application is included in the image



Application Process



Bins / Libs



OS (Base Image)

2 PUBLISH

Upload and publish image

Docker Hub / Registry



Central repository to store and distribute Docker images

3 DOWNLOAD & RUN

Download image and run a container

Docker Host



Container Running
● Running



Image is pulled from the registry and used to run containers on the host

THE FLOW

1



Build
Create image from Dockerfile



2



Publish
Upload image to Docker Hub



3



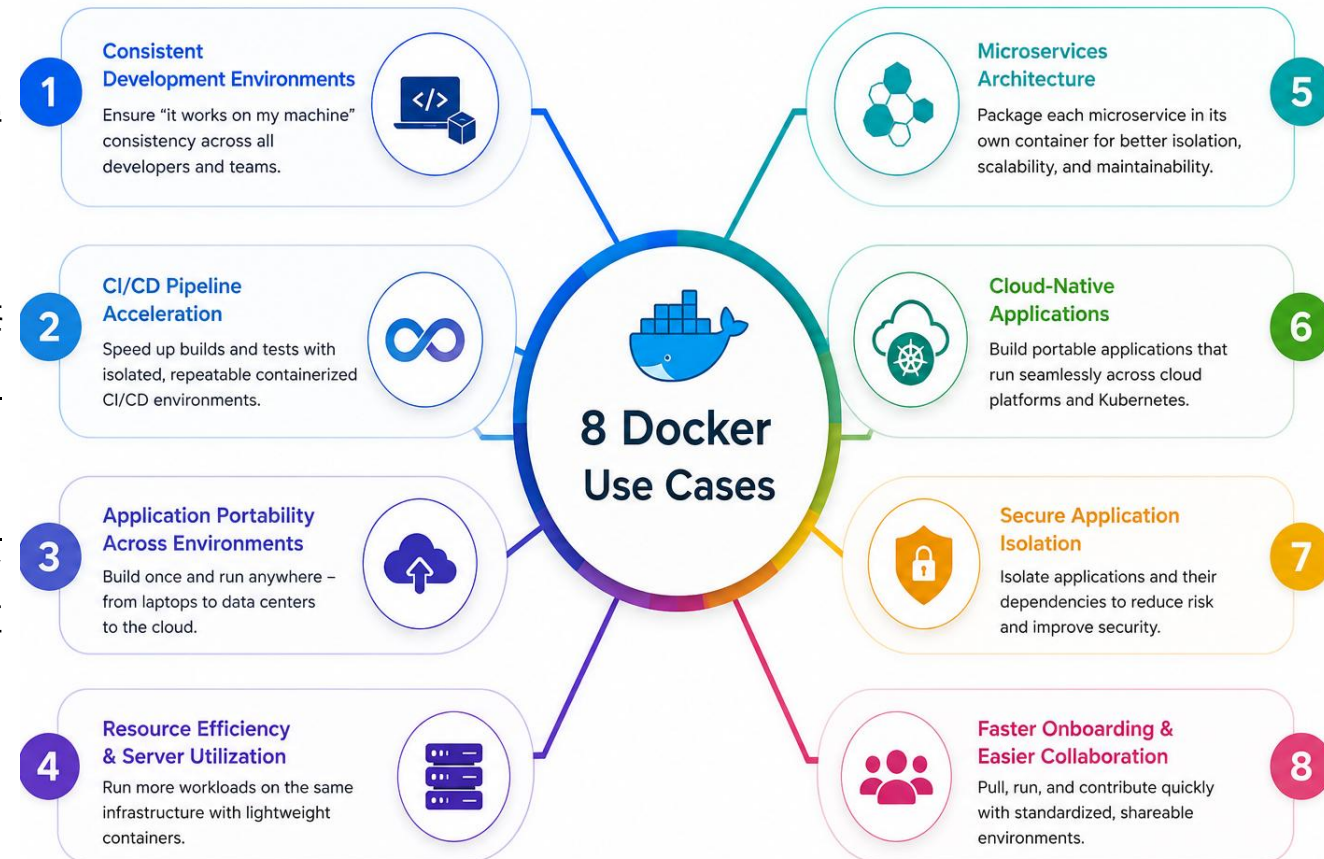
Download & Run
Pull image and run container



- ✓ Consistent environments
- ✓ Easy distribution
- ✓ Versioned & reproducible
- ✓ Runs anywhere

Docker Summary

- A **container** is one or more runtime instances of a Docker image that usually will contain a single app/service. The container is **considered the live artifact being executed in a development machine or the cloud or server**.
- An **image** is an ordered collection of root filesystem changes and the corresponding execution parameters for use within a container runtime. An image typically contains a union of layered filesystems (deltas) stacked on top of each other. An image does not have state and it never changes.
- A **registry** is a service containing repositories of images from one or more development teams. Multiple development teams may also instance multiple registries. The default registry for Docker is the public "Docker Hub" but you will likely have your own private registry network close to your orchestrator to manage and secure your images, and reduce network latency when deploying images.
- The beauty of the images and the registry resides on the possibility for you to store static and immutable application bits including all their dependencies at OS and frameworks level so they can be versioned and deployed in multiple environments providing a consistent deployment unit.

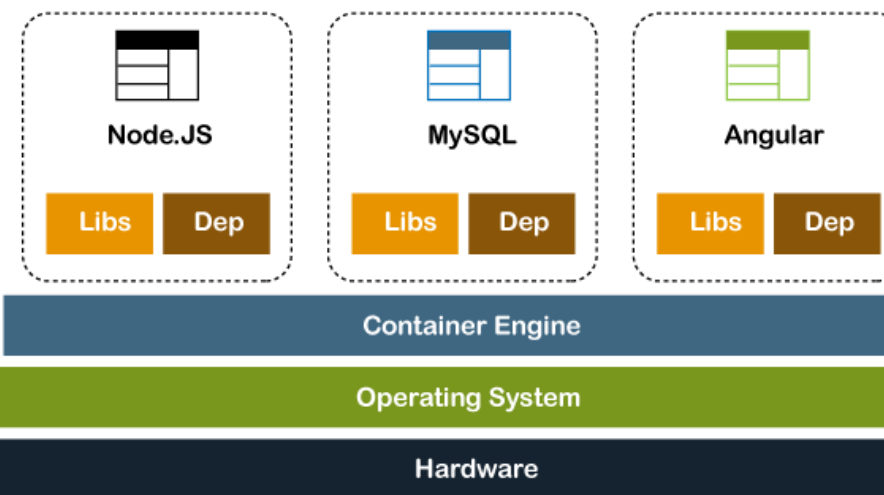


Docker – Video





Container Orchestration

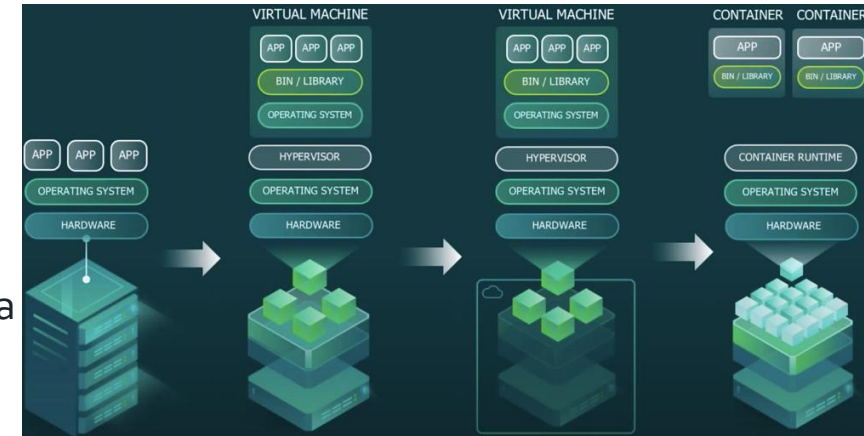


Container Orchestration

Container orchestration is all **about managing the lifecycles of containers**, especially in large, **dynamic environments**.

Software teams use container orchestration to control and automate many tasks:

- Provisioning and deployment of containers
- Scaling up or removing containers to spread application load evenly across host infrastructure
- Movement of containers from one host to another if there is a shortage of resources in a host, or if a host dies
- External exposure of services running in a container with the outside world
- Load balancing of service discovery between containers
- Health monitoring of containers and hosts
- Configuration of an application in relation to the containers running it



Container Orchestration

KEY ORCHESTRATION FEATURES

- Management of container hosts
- Launching set of containers
- Rescheduling failed containers
- Linking containers to support workflows
- Providing connectivity to clients outside the container cluster
- Firewall: control network/port accessibility
- Dynamic scaling of containers: horizontal scaling
 - Scale in/out, add/remove containers
- Load balancing over groups of containers
- Rolling upgrades of containers for application

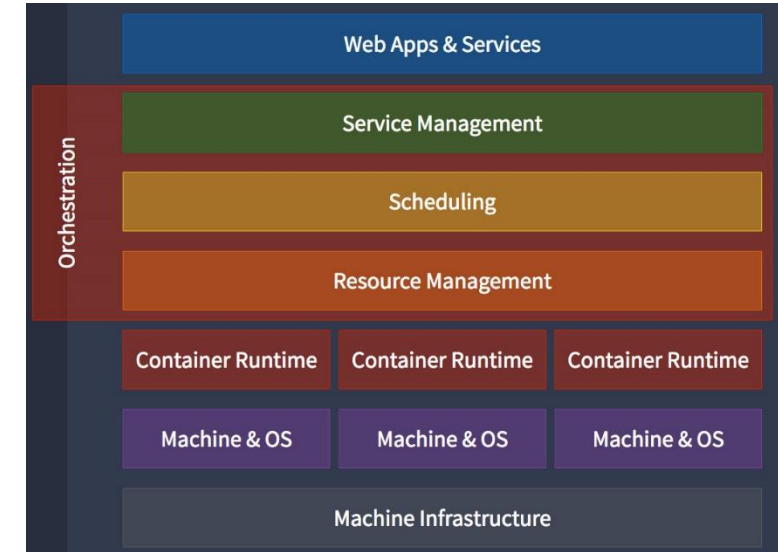
Container Orchestration

Most container orchestration tools support a **declarative configuration model**:

A developer writes a **configuration file** (in YAML or JSON depending on the tool) that defines a desired configuration state, and the orchestration tool runs the file uses its own intelligence to achieve that state. The configuration file typically

- Defines which container images make up the application, and where they are located (in what registry)
- Provisions the containers with storage and other resources
- Defines and secures the network connections between containers
- Specifies versioning (for phased or canary rollouts)

The orchestration tool schedules deployment of the containers (and replicas of the containers, for resiliency) to a host, choosing the best host based on available CPU capacity, memory, or other requirements or constraints specified in the configuration file.



Container orchestration mediates between the apps or services above and the container runtimes below. Three main functional aspects of what they do include:*

- **Service Management:** Labels, groups, namespaces, dependencies, load balancing, readiness checks.
- **Scheduling:** Allocation, replication, resurrection, rescheduling, rolling deployment, upgrades, downgrades.
- **Resource Management:** Memory, CPU, GPU, volumes, ports, IPs.

Imperative vs Declarative Model in IAC

Think about **imperative configuration** like a remote control for a television.

You use the buttons on it to change things until it's how you want it to be. Make the sound louder, change the channel, turn on recording, etc.

Simply put, **imperative** is the "**how**" of configuration paradigms.

In an edge API Gateway, you can go to an interface or a REST API to specifically enter the timeout for routing rules.

First, it's 8 seconds. Then later on, you change it to 5 seconds. You embed how you want it to work into the control flow of your API or the program calling it.

In our analogy, the **declarative model** is like a thermostat. You know that you'll be cold until it's 70F so you set the temperature to 70F and you go about your day.

The thermostat detects its environment and controls the temperature accordingly.

Declarative is **expressing intent** of "**how**" you want your **system to work**.

This model uses a GitOps style approach, because you can write down in your source file that the timeout is 10 seconds and that is committed to source control. Your continuous integration system reads that source file and deploys it to your Kubernetes cluster. So, Git becomes your source of truth.

Kubernetes – Introduction Video



Kubernetes is a self-service Platform-as-a-Service (PaaS) that creates a hardware layer abstraction for development teams. Kubernetes is also extremely portable. It runs on Amazon Web Services (AWS), Microsoft Azure, the Google Cloud Platform (GCP), or in on-premise installations. we can move workloads without having to redesign your applications or completely rethink your infrastructure—which helps you to standardize on a platform and avoid vendor lock-in.

Container Orchestration - Tools

Tools of Container Orchestration



Amazon ECS
FROM AMAZON



Azure Container Services
FROM MICROSOFT



Docker Swarm
DOCKER OPENSOURCE TOOLS



Google Container Engine
FROM GOOGLE CLOUD PLATFORM



Kubernetes
DOCKER OPENSOURCE TOOLS



CoreOS Fleet
FROM COREOS



Mesosphere Marathon
FROM MARATHON



Cloud Foundry's Diego
FROM CLOUD FOUNDRY

- **Docker Swarm:** Provides native clustering functionality for Docker containers, which turns a group of Docker engines into a single, virtual Docker engine.
- **Google Container Engine:** Google Container Engine, built on Kubernetes, lets you run Docker containers on the Google Cloud.
- **Kubernetes:** An orchestration system for Docker containers. It handles scheduling and manages workloads based on user-defined parameters.
- **Mesosphere Marathon:** Marathon is a container orchestration framework for Apache Mesos that is designed to launch long-running applications.
- **Amazon ECS:** The ECS supports Docker containers and lets you run applications on a managed cluster of Amazon EC2 instances.
- **Azure Container Service (ACS):** ACS lets you create a cluster of virtual machines that act as container hosts along with master machines that are used to manage your application containers.
- **Cloud Foundry's Diego:** Container management system that combines a scheduler, runner, and health manager.
- **CoreOS Fleet:** Container management tool that lets you deploy Docker containers on hosts in a cluster as well as distribute services across a cluster.

Q & A.....





BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Credits

*Hwang, Kai; Dongarra, Jack; Fox, Geoffrey C.. Distributed and Cloud Computing: From Parallel Processing to the Internet of Things (Kindle Locations 3532-3533). Elsevier Science. Kindle Edition.